

SA LEVEL UP COURSE – SEP 2025



Microservices Architecture (MSA) From Concepts to Practices

By DungHV6 – FHN.DCS

Course Structure

1) Day 1:

Architectural style evolution

Microservices – A Bird 's Eye View, DDD as an underpinning

2) Day 2:

MSA Concepts, Patterns, Principles, Best Practices, Anti-patterns

Platform, Frameworks, Tools and Techniques to realize MSA

3) Day 3:

-Implement MSA with 2 MSA platforms (Spring Platform, Kubernetes/Openshift)

-Microservices' role in AI Era

Day 2



Intentionally left blank

Core MSA Patterns & Best Practices



To address afore-mentioned challenges, patterns below were born. An application is MSA-compliant if it adheres to them:

- Autonomy
- Service Registration and Discovery (Service Registry)
- API Management
- App-level, service-level Security (with SSO, mTLS)
- Externalized and centralized Configs Management
- Inter-service communications (sync, async with different protocols, http, tcp, gRPC)
- Scalability (Autoscaling)
- Resilience (Timeout, Retries, Circuit Breaking, Bulkheading)
- Observability (Centralized and Aggregated Logging, Distributed Tracing, Monitoring)
- Containerization (Dockerization)

There are several tech stacks out there that realize above patterns. At the first phase, for demonstration purpose, we decided on Spring Cloud, Netflix OSS stacks.

12-Factor App Manifesto by Heroku



- Codebase

Single self-contained codebase with Http/Websocket/gRPC servers embedded into the codebase

- Configuration (must have)

Externalize app configs into config services (i.e Spring Config Server, Kubernetes ConfigMaps, Secrets) and vaults (Hashicorp Vault, AWS KMS, AWS Secret Manager...)

Reference for 12 best practices: <https://12factor.net/>

Beyond 12-Factor App Manifesto



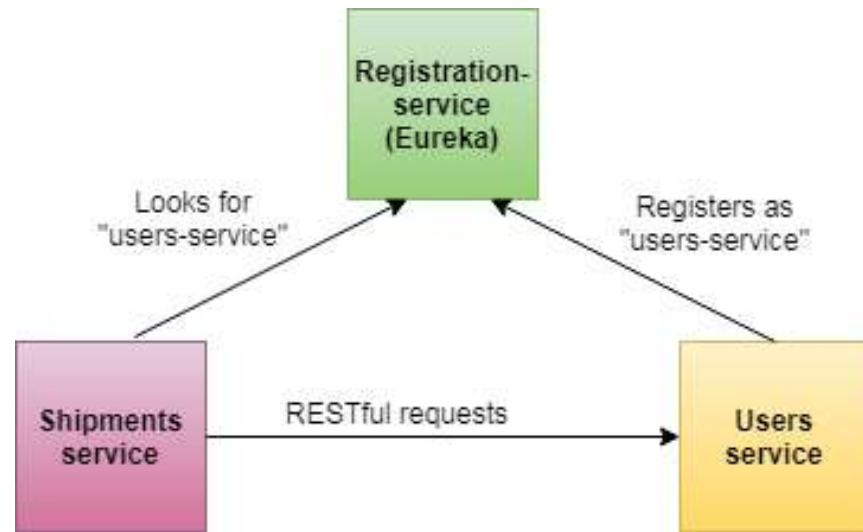
- API First Approach (w/ OpenAPI 3, GraphQL, gRPC...)
- Observability (Distributed Logging, Tracing, Monitoring) with OpenTelemetry
- Security (OWASP-compliant)

Pattern 1 - Service Registry

- Helps microservices communicate with each other in physical location transparency manner

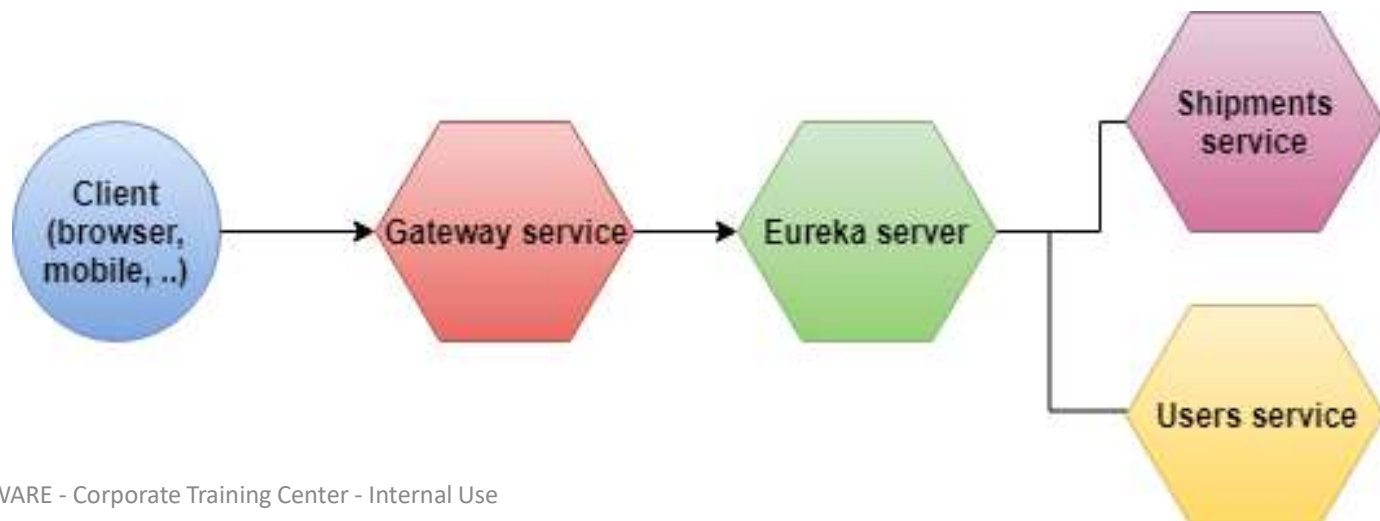
Service Registry examples:

- Spring Eureka
- Kubernetes Services (strongly recommended)

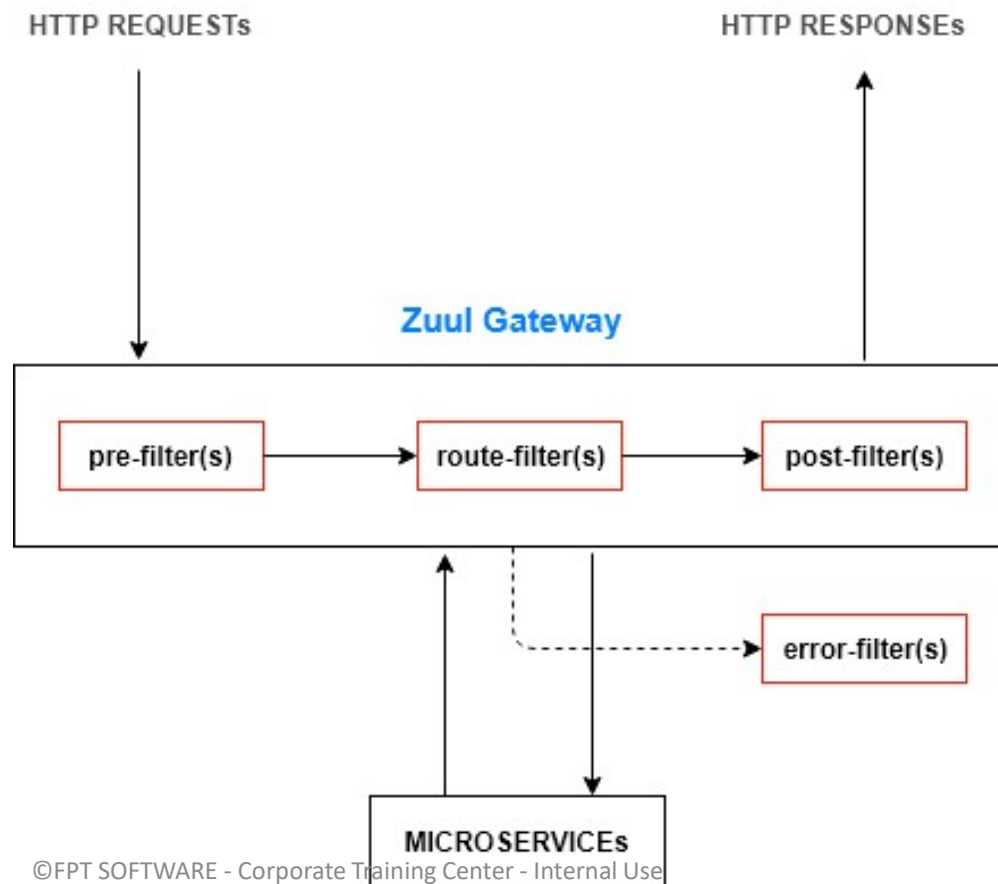


Pattern 2 - API Gateway

- API Gateway provides dynamic routing, monitoring, resiliency, security, APIs management, analytics, throttling.
- Examples: Zuul
- Recommended: Kong, Google APIGee, AWS API Gateway, Azure API Gateway



API Gateway

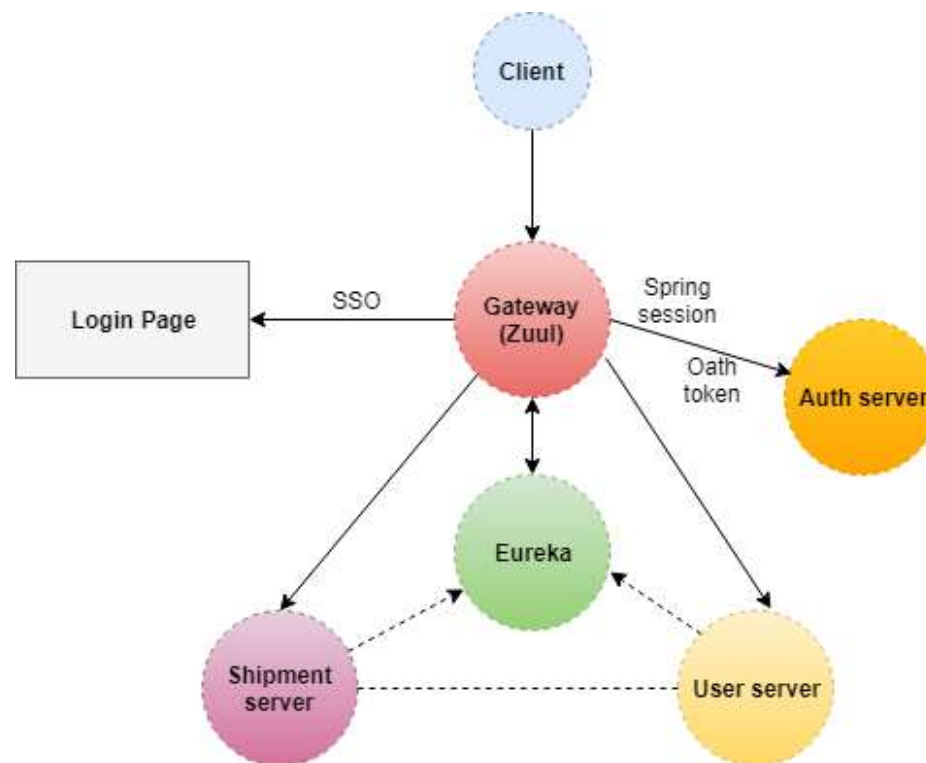


Pattern 3 - Single-sign on and Security



- Microservices system is protected by SSO OAuth2 behind Zuul gateway. Requests to the microservices and authorization server are proxied by the gateway. First request is redirected to the login page. Next step is to obtain OAuth2 authorization token by calling authorization server endpoints via gateway. Finally, you can call concrete microservice providing OAuth2 token as a bearer in Authorization HTTP request header.

Single-sign on and Security



Pattern 4 - Inter-service Communications



- There exist 2 main types of inter-service communication, sync and async
- Sync communication occurs over Http/s and by means of simple and lightweight RESTFul Web Services. A client needs to wait for server 's processing until a response returns, then client continues to proceed. gRPC is another protocol for this kind of communication
- Asyn communication occurs over TCP/IP with the aid of message brokers (Message-oriented Middlewares: RabbitMQ, Kafka). A client just publishes messages and continues its execution without having to wait for server 's response.

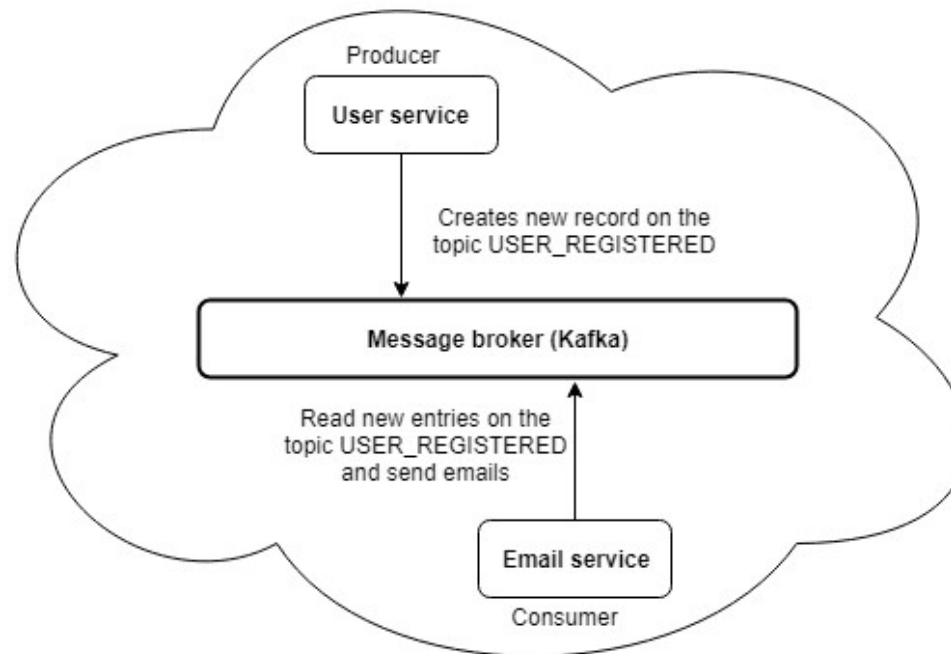
Messaging

- Asynchronous kind of inter-service communication, in addition to synchronous one (RESTful request/response).
- Use message brokers (ActiveMQ, RabbitMQ, Pulsar, Kafka, Azure EventHub, AWS Kinesis Data Streams, Google PubSub...).
- Helps solve problem of microservices: distributed data management.

QoS Notes:

Configure Message Brokers in combination with Publisher/Consumer APIs so as to guarantee QoS like At most once, At least once, Exactly once

Messaging



Pattern 5 - Resilience



- Child Patterns:
Timeout, retry, bulkheading, circuit breaking

Timeout

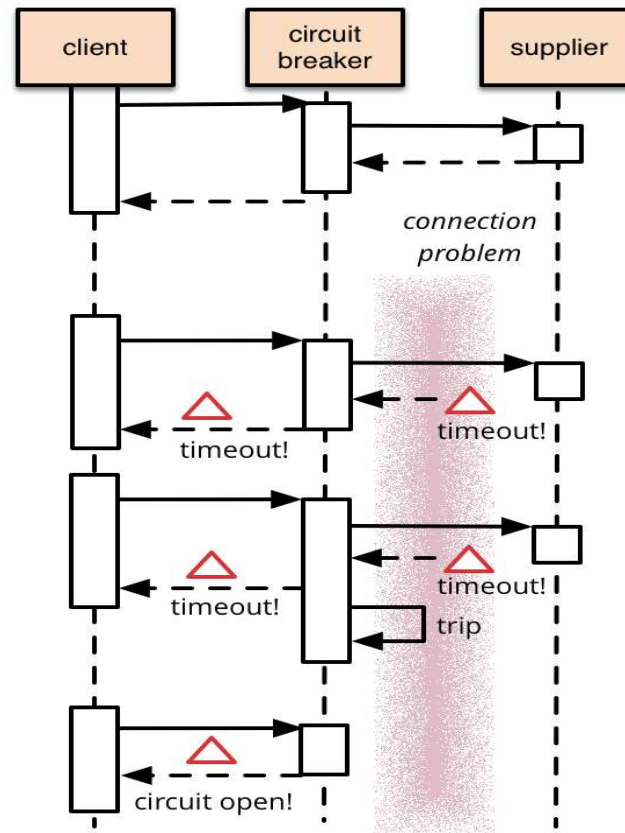
- At app/service level, when a caller service invokes another service/system, timeout is used to decide when to stop waiting for the response from from the called service/system. The timeout value is set so that when the timeout is reached, some specific logics are triggered in reaction to that event.

Circuit Breaker



- Circuit breaker pattern ensures stable performance in your microservices by monitoring for failures and providing an alternate service or error message.
- Once the failures reach a certain threshold, the circuit breaker trips, and all further calls to the circuit breaker return with an error, without the protected call being made at all.

Circuit Breaking



- This page is intentionally left blank

Pattern 6 – OTel-compliant Observability



- 3 Pillars of observability are logging, tracing and monitoring
- Realizing Observability requires synergy amongst SA, development, devops teams

SA: Design diagrams, solutions, propose tech stacks

Developer: Add dependencies, log statements, custom spans, monitoring metrics configurations to microservice codebase

Devops: provision infrastructure, related cloud services for web/database servers

Special Notes:

Selected Observability stacks need to implement and be compliant with Otel specifications (<https://opentelemetry.io>)

SA/Devops need to factor clients' infrastructural constraints into selection criteria. If customers already have commercial products (Splunk, New Relic, Datadog...), observability solution need to prioritize these.

Pattern 6 – Observability (Cont.)



Special Notes on Why & When to apply:

- Distributed Logging, tracing, monitoring tools are instrumental in debugging and detect bugs, performance issues in business use cases inside a project
- Need to apply soon after a project starts (ideal before development starts)

Recommended Combos

- Distributed Logging: Slf4j (java lib), FluentD(transporter), ElasticSearch(storage), Kibana (dashboard)
- Distributed Tracing: OpenTelemetry, Zipkin/Jaeger (Backend)
- Monitoring: OpenTelemetry, Prometheus, Grafana

Distributed Tracing w/ Open Trace Jaeger

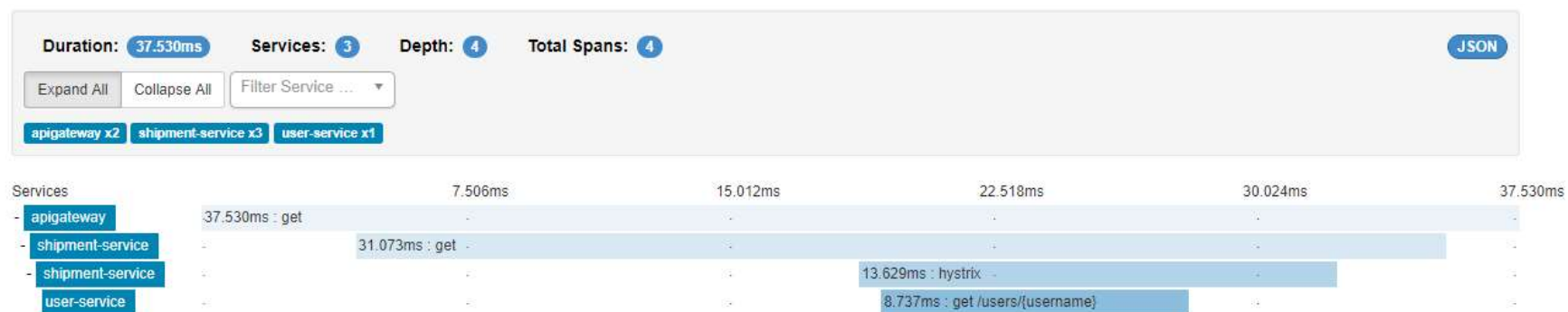


- In the microservices world, a user action on UI may invoke one microservice API endpoint, which in turn invoke another microservice endpoint.
- Provides the distributed tracing capabilities and we can also export these trace information to Zipkin to visualize the call traces.

Zipkin

Zipkin Investigate system behavior Find a trace Dependencies

Go to trace



Monitoring w/ Prometheus and Grafana stack



- The purpose of monitoring is to help admins/operators check the health of all microservices instances and alert them before or at the time incidents occur inside a MSA application for their preventative and corrective actions

Prometheus



- A tool that gathers app metrics, statistics about microservices health from microservices instrumentation endpoints, saves them into its own data store for consumption by Graphana

Grafana



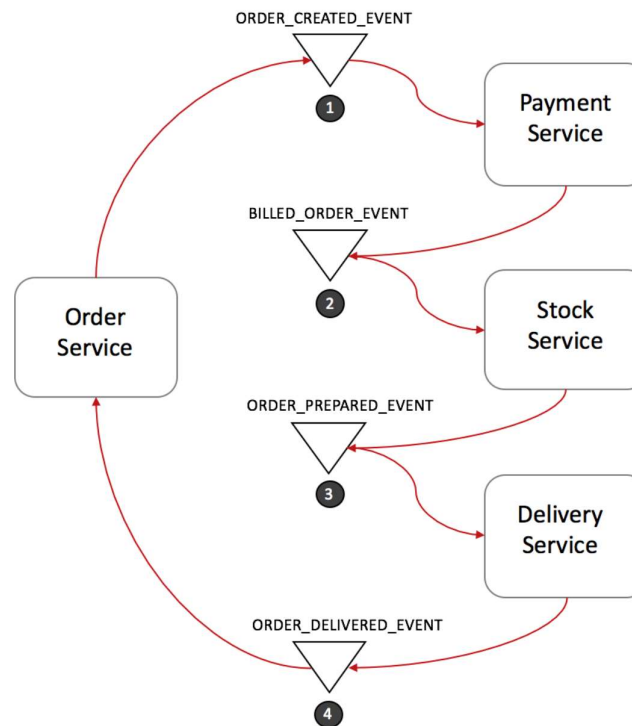
- A Dashboard that fetches data from Prometheus metrics store, displays them in form of charts, graphs, diagrams for visualization, alerts administrators, stakeholders about incidents...

Distributed Transaction Management



- Data consistency for microservices in the distributed and heterogeneous environment is guaranteed by Saga pattern that is a sequence of local transactions. Each of them modifies data states using traditional ACID mechanism. Local transactions are coordinated using async messaging.
- After a saga participant commits, it fires an event to the next one. This loops until the last one reached.
- Sagas use compensating transactions to roll back data changes (undo changes)
- There exists 2 ways to coordinate sagas, choreography-based and orchestration-based ones.

Tx Management with Choreography-based Saga

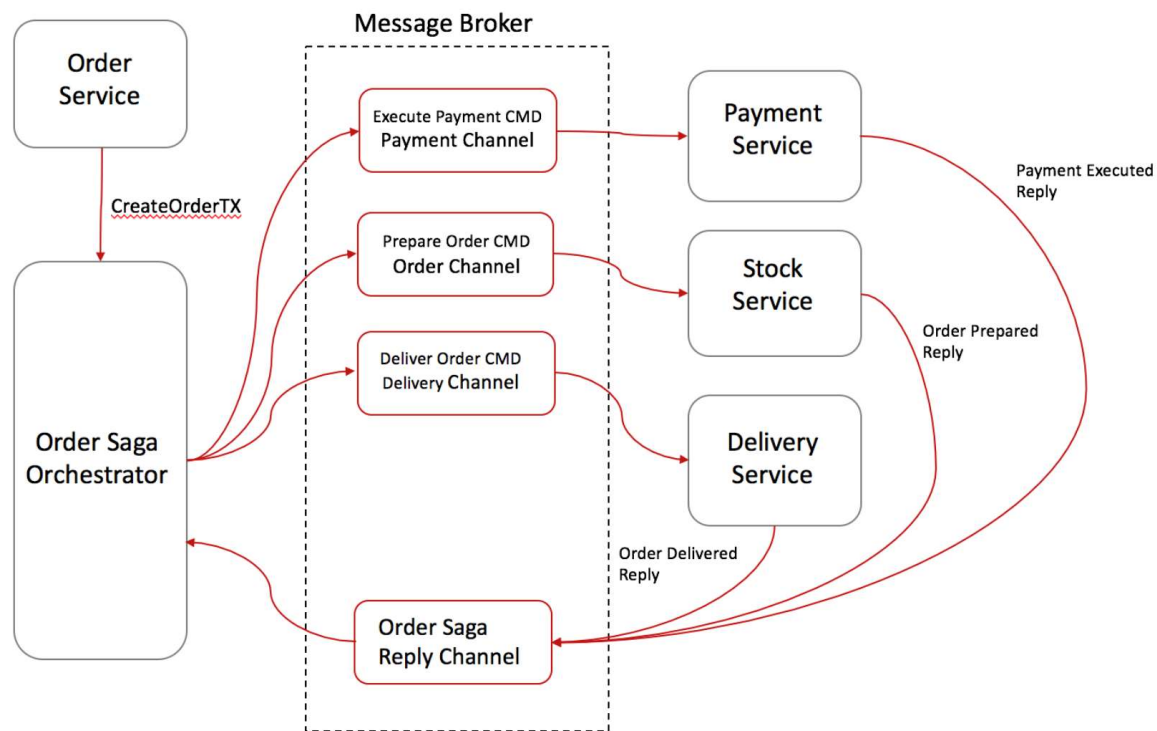


Source:

<https://blog.couchbase.com/saga-pattern-implement-business-transactions-using-microservices-part/>

©FPT SOFTWARE - Corporate Training Center - Internal Use

Tx Management with Command/Orchestration-based Saga



Source: ©FPT SOFTWARE - Corporate Training Center - Internal Use

<https://blog.couchbase.com/saga-pattern-implement-business-transactions-using-microservices-part-2/>

CQRS & Event Sourcing



Refer to "CQRS/ES with Axon" presentation slides

References

MSA Topics:

Domain Driven Design, Microservices Patterns,

Must-read Web Pages:

<https://microservices.io>

Recommended Books:

- Implementing Domain Driven Design
- Patterns, Principles and Practices of Domain-Driven Design (strongly recommended)
- Tackle complexity
- Microservices Patterns
- Spring Microservice in action
- Kubernetes/Openshift in action
- Istio in action
- Microservices in .NET

Thank you for joining us!

